레지스트리 서비스 (Registry Service)

소개

레지스트리 서비스는 Oasis Core 네트워크에서 엔티티(Entity), 노드(Node), 런타임(Runtime) 등록 시스템을 관리하는 핵심 서비스입니다. 이 서비스는 네트워크 참여자들의 신원 확인, 메타데이터 관리, 공개키 배포 등의 기능을 제공합니다.

주요 특징

- **엔티티 관리**: 네트워크 참여 주체의 등록과 관리
- **노드 등록**: 다양한 역할의 노드 등록과 검증
- **런타임 등록**: 컴퓨트 및 키매니저 런타임 등록
- **메타데이터 관리**: 노드와 런타임의 상세 정보 저장
- **공개키 배포**: 암호화 통신을 위한 공개키 관리

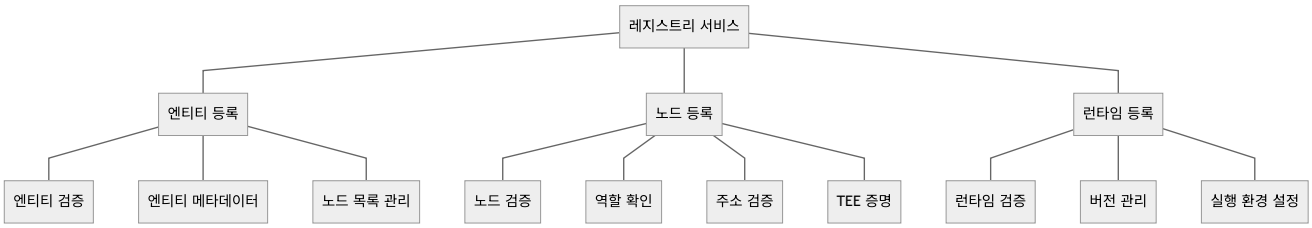
관련 프로세스

- **합의 프로세스**: 밸리데이터 노드 등록과 검증
- **스테이킹 서비스**: 노드 등록을 위한 스테이킹 요구사항
- **스케줄러 서비스**: 등록된 노드 기반 위원회 구성
- **런타임 계층**: 등록된 런타임 실행과 관리

아키텍처

전체 구조

레지스트리 서비스는 세 가지 주요 등록 유형을 관리합니다.



서비스 인터페이스

```
// Backend는 레지스트리 서비스의 백엔드 인터페이스입니다
type Backend interface {
// 엔티티 관련 메서드
GetEntity(ctx context.Context, query *IDQuery) (*entity.Entity, error)
GetEntities(ctx context.Context, height  int64) ([]*entity.Entity, error)
WatchEntities(ctx context.Context) (<-chan *EntityEvent, pubsub.ClosableSubscrip
// 노드 관련 메서드
GetNode(ctx context.Context, query *IDQuery) (*node.Node, error)
GetNodes(ctx context.Context, height  int64) ([]*node.Node, error)
WatchNodes(ctx context.Context) (<-chan *NodeEvent, pubsub.ClosableSubscription,
// 런타임 관련 메서드
GetRuntime(ctx context.Context, query *GetRuntimeQuery) (*Runtime, error)
```

```go
GetRuntimes(ctx context.Context, query *GetRuntimesQuery) ([]*Runtime, error)

WatchRuntimes(ctx context.Context) (<-chan *Runtime, pubsub.ClosableSubscription

// 상태 관리

StateToGenesis(ctx context.Context, height  int64) (*Genesis, error)

GetEvents(ctx context.Context, height  int64) ([]*Event, error)

Cleanup()

}
```

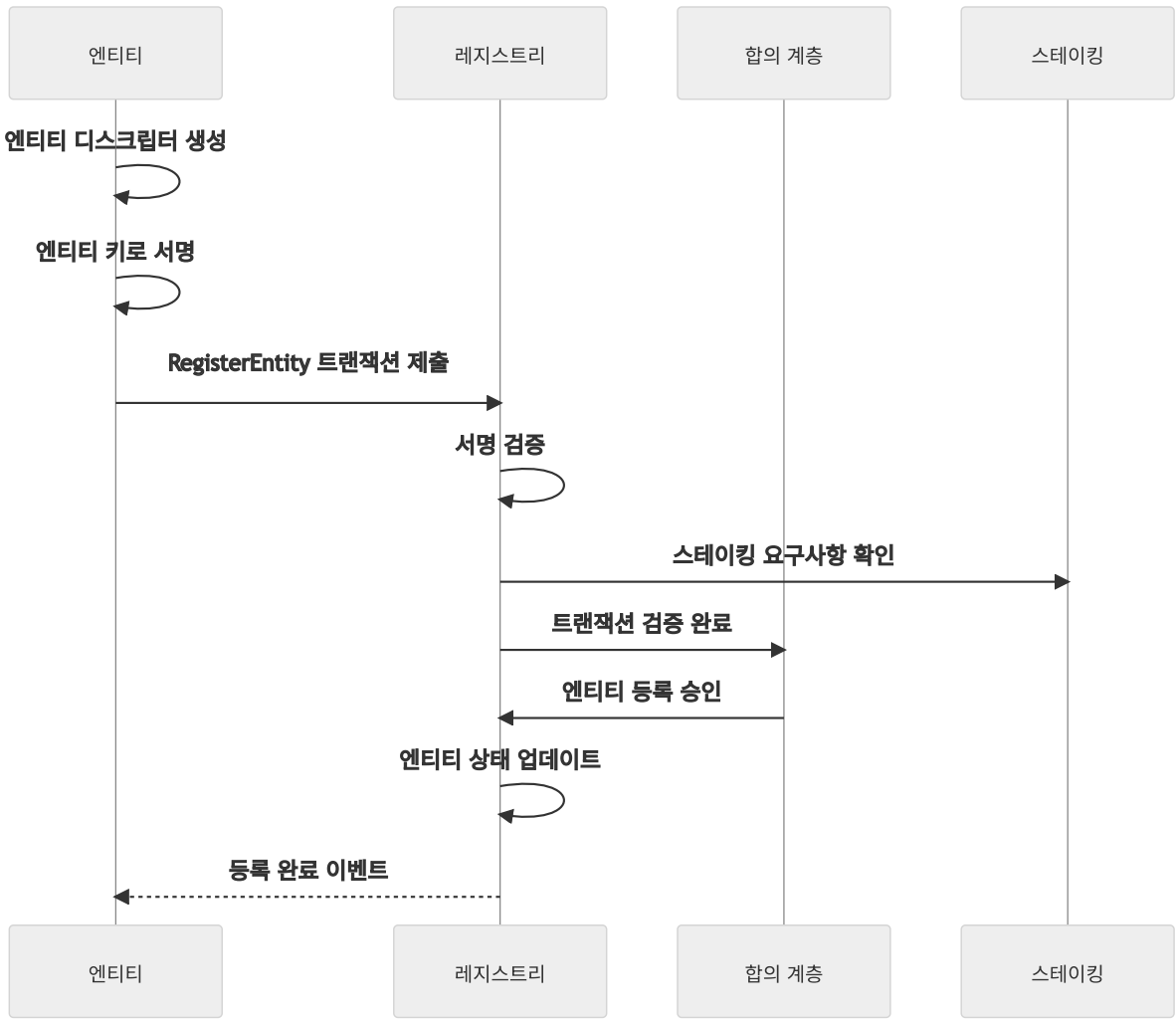
엔티티 등록 프로세스

1. 엔티티 구조

```go
// Entity는 네트워크 참여 엔티티를 나타냅니다

type Entity struct {

cbor.Versioned

// ID는 엔티티의 공개키입니다

ID signature.PublicKey `json:"id"`

// Nodes는 이 엔티티가 운영하는 노드 목록입니다

Nodes []signature.PublicKey `json:"nodes,omitempty"`

// AllowEntitySignedNodes는 엔티티 키로 서명된 노드를 허용할지 여부입니다

AllowEntitySignedNodes  bool  `json:"allow_entity_signed_nodes,omitempty"`

}
```

```go
// SignedEntity는 서명된 엔티티입니다

type SignedEntity struct {

cbor.Versioned

// Signed는 서명된 엔티티 디스크립터입니다

Signed signature.Signed `json:"signed"`

}
```

2. 엔티티 등록 과정



3. 엔티티 검증 로직

```
// VerifyRegisterEntityArgs는 엔티티 등록 인수를 검증합니다

func  VerifyRegisterEntityArgs(

logger *logging.Logger,

sigEnt *entity.SignedEntity,
```

```go
isGenesis, isSanityCheck  bool,
) (*entity.Entity, error) {
var  ent entity.Entity
if  sigEnt == nil {
return  nil, ErrInvalidArgument
}
// 1. 서명 검증
if  err := sigEnt.Open(RegisterEntitySignatureContext, &ent);  err != nil {
logger.Error("엔티티 서명 검증 실패", "err", err)
return  nil, ErrInvalidSignature
}
// 2. 엔티티 ID 검증
if !ent.ID.IsValid() {
return  nil, ErrInvalidArgument
}
// 3. 노드 목록 검증
if  err := verifyEntityNodes(&ent);  err != nil {
return  nil, err
}
return &ent, nil
}


func  verifyEntityNodes(ent *entity.Entity) error {
```

```go
// 중복 노드 확인
nodeSet := make(map[signature.PublicKey]bool)
for _, nodeID := range ent.Nodes {
if nodeSet[nodeID] {
return fmt.Errorf("중복 노드 ID: %s", nodeID)
}
nodeSet[nodeID] = true
}
return nil
}
```

노드 등록 프로세스

1. 노드 구조

```go
// Node는 네트워크 노드를 나타냅니다
type Node struct {
cbor.Versioned
// ID는 노드의 공개키입니다
ID signature.PublicKey `json:"id"`
// EntityID는 노드를 소유한 엔티티의 ID입니다
EntityID signature.PublicKey `json:"entity_id"`
// Expiration은 노드 등록 만료 에포크입니다
```

```
Expiration  uint64  `json:"expiration"`

// TLS는 노드의 TLS 정보입니다

TLS TLSInfo `json:"tls"`

// P2P는 노드의 P2P 정보입니다

P2P P2PInfo `json:"p2p"`

// Consensus는 노드의 합의 정보입니다

Consensus ConsensusInfo `json:"consensus"`

// VRF는 노드의 VRF 정보입니다 (선택사항)

VRF *VRFInfo `json:"vrf,omitempty"`

// Runtimes는 노드가 지원하는 런타임 목록입니다

Runtimes []*Runtime `json:"runtimes,omitempty"`

// Roles는 노드의 역할입니다

Roles RolesMask `json:"roles"`

}


// TLSInfo는 노드의 TLS 정보입니다

type TLSInfo struct {

PubKey signature.PublicKey `json:"pub_key"`

}


// P2PInfo는 노드의 P2P 정보입니다

type P2PInfo struct {

ID signature.PublicKey `json:"id"`
```

```
Addresses []Address `json:"addresses,omitempty"`
}

// ConsensusInfo는 노드의 합의 정보입니다
type ConsensusInfo struct {
ID signature.PublicKey `json:"id"`
Addresses []ConsensusAddress `json:"addresses,omitempty"`
}
```

2. 노드 역할 정의

```
// RolesMask는 노드 역할의 비트마스크입니다
type RolesMask uint32

const (
// RoleComputeWorker는 컴퓨트 워커 역할입니다
RoleComputeWorker RolesMask = 1 << 0
// RoleObserver는 옵저버 역할입니다
RoleObserver RolesMask = 1 << 1
// RoleKeyManager는 키 매니저 역할입니다
RoleKeyManager RolesMask = 1 << 2
// RoleValidator는 밸리데이터 역할입니다
RoleValidator RolesMask = 1 << 3
```

```go
// RoleConsensusRPC는 합의 RPC 역할입니다
RoleConsensusRPC RolesMask = 1 << 4
// RoleStorageRPC는 스토리지 RPC 역할입니다
RoleStorageRPC RolesMask = 1 << 5
)

// HasRoles는 노드가 지정된 역할을 가지고 있는지 확인합니다
func (r RolesMask) HasRoles(roles RolesMask) bool {
return (r & roles) == roles
}

// IsSingleRole은 노드가 단일 역할만 가지고 있는지 확인합니다
func (r RolesMask) IsSingleRole() bool {
return  r != 0 && (r&(r-1)) == 0
}
```

3. 노드 등록 검증

```go
// VerifyRegisterNodeArgs는 노드 등록 인수를 검증합니다
func  VerifyRegisterNodeArgs(
ctx context.Context,
params *ConsensusParameters,
logger *logging.Logger,
```

```go
sigNode *node.MultiSignedNode,
entity *entity.Entity,
now beacon.EpochTime,
isGenesis  bool,
isSanityCheck  bool,
nodeLookup NodeLookup,
runtimeLookup RuntimeLookup,
) (*node.Node, []*Runtime, error) {
// 1. 기본 검증
if  sigNode == nil {
return  nil, nil, ErrInvalidArgument
}
var  n node.Node
if  err := sigNode.Open(RegisterNodeSignatureContext, &n);  err != nil {
logger.Error("노드 서명 검증 실패", "err", err)
return  nil, nil, ErrInvalidSignature
}
// 2. 엔티티 검증
if !n.EntityID.Equal(entity.ID) {
return  nil, nil, ErrBadEntityForNode
}
// 3. 만료 시간 검증
if !isGenesis && n.Expiration <= uint64(now) {
```

```go
return nil, nil, ErrNodeExpired
}
// 4. 역할 검증
if err := verifyNodeRoles(&n, params); err != nil {
return nil, nil, err
}
// 5. 주소 검증
if err := verifyNodeAddresses(&n); err != nil {
return nil, nil, err
}
// 6. 런타임 검증
runtimes, err := verifyNodeRuntimes(ctx, &n, runtimeLookup)
if err != nil {
return nil, nil, err
}
return &n, runtimes, nil
}


func verifyNodeRoles(n *node.Node, params *ConsensusParameters) error {
// 역할이 비어있으면 안됨
if n.Roles == 0 {
return fmt.Errorf("노드 역할이 지정되지 않음")
}
```

```go
// 유효하지 않은 역할 조합 확인
if n.Roles.HasRoles(RoleValidator) && n.Roles.HasRoles(RoleKeyManager) {
return fmt.Errorf("밸리데이터와 키매니저 역할은 동시에 가질 수 없음")
}
return nil
}


func verifyNodeAddresses(n *node.Node) error {
// P2P 주소 검증
for _, addr := range n.P2P.Addresses {
if err := addr.Validate(); err != nil {
return fmt.Errorf("잘못된 P2P 주소: %w", err)
}
}
// 합의 주소 검증
for _, addr := range n.Consensus.Addresses {
if err := addr.Validate(); err != nil {
return fmt.Errorf("잘못된 합의 주소: %w", err)
}
}
return nil
}
```

런타임 등록 프로세스

1. 런타임 구조

```
// Runtime은 등록된 런타임을 나타냅니다
type Runtime struct {
cbor.Versioned
// ID는 런타임의 고유 식별자입니다
ID common.Namespace `json:"id"`
// EntityID는 런타임을 소유한 엔티티의 ID입니다
EntityID signature.PublicKey `json:"entity_id"`
// Genesis는 런타임 제네시스 정보입니다
Genesis RuntimeGenesis `json:"genesis"`
// Kind는 런타임 유형입니다
Kind RuntimeKind `json:"kind"`
// TEEHardware는 필요한 TEE 하드웨어입니다
TEEHardware node.TEEHardware `json:"tee_hardware"`
// Deployments는 런타임 배포 정보입니다
Deployments []*VersionInfo `json:"deployments,omitempty"`
// KeyManager는 키 매니저 런타임 ID입니다 (선택사항)
KeyManager *common.Namespace `json:"key_manager,omitempty"`
// Executor는 실행자 매개변수입니다
Executor ExecutorParameters `json:"executor,omitempty"`
```

```go
// TxnScheduler는 트랜잭션 스케줄러 매개변수입니다
TxnScheduler TxnSchedulerParameters `json:"txn_scheduler,omitempty"`
// Storage는 스토리지 매개변수입니다
Storage StorageParameters `json:"storage,omitempty"`
// AdmissionPolicy는 노드 허용 정책입니다
AdmissionPolicy RuntimeAdmissionPolicy `json:"admission_policy"`
// Constraints는 스케줄링 제약사항입니다
Constraints  map[scheduler.CommitteeKind]map[scheduler.Role]SchedulingConstraint
// Staking은 스테이킹 매개변수입니다
Staking RuntimeStakingParameters `json:"staking,omitempty"`
// GovernanceModel은 거버넌스 모델입니다
GovernanceModel RuntimeGovernanceModel `json:"governance_model"`
}


// RuntimeKind는 런타임 기능을 나타냅니다
type RuntimeKind uint32


const (
// KindInvalid는 유효하지 않은 런타임입니다
KindInvalid RuntimeKind = 0
// KindCompute는 컴퓨트 런타임입니다
KindCompute RuntimeKind = 1
// KindKeyManager는 키 매니저 런타임입니다
```

```
KindKeyManager RuntimeKind = 2

)
```

2. 런타임 허용 정책

```
// RuntimeAdmissionPolicy는 런타임에 등록할 수 있는 노드를 지정합니다

type RuntimeAdmissionPolicy struct {

AnyNode *AnyNodeRuntimeAdmissionPolicy `json:"any_node,omitempty"`

EntityWhitelist *EntityWhitelistRuntimeAdmissionPolicy `json:"entity_whitelist,o

}


// AnyNodeRuntimeAdmissionPolicy는 모든 노드의 등록을 허용합니다

type AnyNodeRuntimeAdmissionPolicy struct{}


// EntityWhitelistRuntimeAdmissionPolicy는 화이트리스트된 엔티티의 노드만 허용합니다

type EntityWhitelistRuntimeAdmissionPolicy struct {

Entities  map[signature.PublicKey]EntityWhitelistConfig `json:"entities"`

}


// EntityWhitelistConfig는 엔티티별 화이트리스트 설정입니다

type EntityWhitelistConfig struct {

// MaxNodes는 엔티티가 특정 역할로 등록할 수 있는 최대 노드 수입니다

MaxNodes  map[scheduler.Role]uint16  `json:"max_nodes,omitempty"`
```

```
}
```

3. 런타임 등록 검증

```
// VerifyRegisterRuntimeArgs는 런타임 등록 인수를 검증합니다
func VerifyRegisterRuntimeArgs(
ctx context.Context,
params *ConsensusParameters,
logger *logging.Logger,
rt *Runtime,
entity *entity.Entity,
isGenesis bool,
isSanityCheck bool,
runtimeLookup RuntimeLookup,
) error {
// 1. 기본 검증
if rt == nil {
return ErrInvalidArgument
}
// 2. 런타임 ID 검증
if !rt.ID.IsValid() {
return ErrInvalidArgument
}
```

```go
// 3. 엔티티 검증
if !rt.EntityID.Equal(entity.ID) {
return ErrBadEntityForRuntime
}
// 4. 런타임 종류별 검증
switch rt.Kind {
case KindCompute:
return VerifyRegisterComputeRuntimeArgs(ctx, logger, rt, runtimeLookup)
case KindKeyManager:
return VerifyRegisterKeyManagerRuntimeArgs(ctx, logger, rt, runtimeLookup)
default:
return fmt.Errorf("지원되지 않는 런타임 종류: %d", rt.Kind)
}
}

// VerifyRegisterComputeRuntimeArgs는 컴퓨트 런타임 등록을 검증합니다
func VerifyRegisterComputeRuntimeArgs(
ctx context.Context,
logger *logging.Logger,
rt *Runtime,
runtimeLookup RuntimeLookup,
) error {
// 키 매니저 검증 (설정된 경우)
```

```go
if rt.KeyManager != nil {
kmRt, err := runtimeLookup.Runtime(ctx, *rt.KeyManager)
if err != nil {
return fmt.Errorf("키 매니저 런타임을 찾을 수 없음: %w", err)
}
if kmRt.Kind != KindKeyManager {
return fmt.Errorf("지정된 런타임이 키 매니저가 아님")
}
// TEE 하드웨어 호환성 확인
if rt.TEEHardware != kmRt.TEEHardware {
return fmt.Errorf("TEE 하드웨어 불일치")
}
}
// 실행자 매개변수 검증
if err := verifyExecutorParameters(&rt.Executor); err != nil {
return err
}
// 트랜잭션 스케줄러 매개변수 검증
if err := verifyTxnSchedulerParameters(&rt.TxnScheduler); err != nil {
return err
}
return nil
```

```
}
```

상태 관리

1. 레지스트리 상태

```
// State는 레지스트리의 불변 상태입니다
type ImmutableState struct {
is *abciAPI.ImmutableState
}

// GetEntity는 엔티티를 조회합니다
func (s *ImmutableState) GetEntity(ctx context.Context, id signature.PublicKey)
value, err := s.is.Get(ctx, entityKeyFmt.Encode(&id))
if err != nil {
return nil, abciAPI.UnavailableStateError(err)
}
if value == nil {
return nil, ErrNoSuchEntity
}
var ent entity.Entity
if err = cbor.Unmarshal(value, &ent); err != nil {
return nil, abciAPI.UnavailableStateError(err)
```

```go
}
return &ent, nil
}


// GetNode는 노드를 조회합니다
func (s *ImmutableState) GetNode(ctx context.Context, id signature.PublicKey) (*
value, err := s.is.Get(ctx, nodeKeyFmt.Encode(&id))
if err != nil {
return nil, abciAPI.UnavailableStateError(err)
}
if value == nil {
return nil, ErrNoSuchNode
}
var n node.Node
if err = cbor.Unmarshal(value, &n); err != nil {
return nil, abciAPI.UnavailableStateError(err)
}
return &n, nil
}


// GetRuntime은 런타임을 조회합니다
func (s *ImmutableState) GetRuntime(ctx context.Context, id common.Namespace) (*
value, err := s.is.Get(ctx, runtimeKeyFmt.Encode(&id))
```

```go
if err != nil {
return nil, abciAPI.UnavailableStateError(err)
}
if value == nil {
return nil, ErrNoSuchRuntime
}
var rt Runtime
if err = cbor.Unmarshal(value, &rt); err != nil {
return nil, abciAPI.UnavailableStateError(err)
}
return &rt, nil
}
```

2. 이벤트 시스템

```go
// EntityEvent는 엔티티 등록 변경을 알리는 이벤트입니다
type EntityEvent struct {
Entity *entity.Entity `json:"entity"`
IsRegistration bool `json:"is_registration"`
}

// NodeEvent는 노드 등록 변경을 알리는 이벤트입니다
type NodeEvent struct {
```

```
Node *node.Node `json:"node"`

IsRegistration  bool  `json:"is_registration"`

}



// RuntimeStartedEvent는 런타임 시작을 알리는 이벤트입니다

type RuntimeStartedEvent struct {

Runtime *Runtime `json:"runtime"`

}



// RuntimeSuspendedEvent는 런타임 중단을 알리는 이벤트입니다

type RuntimeSuspendedEvent struct {

RuntimeID common.Namespace `json:"runtime_id"`

}
```

트랜잭션 처리

1. 트랜잭션 유형

```
const (

// MethodRegisterEntity는 엔티티 등록 메서드입니다

MethodRegisterEntity = transaction.MethodName("registry.RegisterEntity")

// MethodDeregisterEntity는 엔티티 등록 해제 메서드입니다

MethodDeregisterEntity = transaction.MethodName("registry.DeregisterEntity")
```

```
// MethodRegisterNode는 노드 등록 메서드입니다
MethodRegisterNode = transaction.MethodName("registry.RegisterNode")
// MethodUnfreezeNode는 노드 동결 해제 메서드입니다
MethodUnfreezeNode = transaction.MethodName("registry.UnfreezeNode")
// MethodRegisterRuntime은 런타임 등록 메서드입니다
MethodRegisterRuntime = transaction.MethodName("registry.RegisterRuntime")
)

// 트랜잭션 생성 함수들
func NewRegisterEntityTx(nonce uint64, fee *transaction.Fee, sigEnt *entity.Si
return transaction.NewTransaction(nonce, fee, MethodRegisterEntity, sigEnt)
}

func NewRegisterNodeTx(nonce uint64, fee *transaction.Fee, sigNode *node.Multi
return transaction.NewTransaction(nonce, fee, MethodRegisterNode, sigNode)
}

func NewRegisterRuntimeTx(nonce uint64, fee *transaction.Fee, rt *Runtime) *tr
return transaction.NewTransaction(nonce, fee, MethodRegisterRuntime, rt)
}
```

2. 트랜잭션 실행

```go
// ExecuteTx는 레지스트리 트랜잭션을 실행합니다
func (app *registryApplication) ExecuteTx(ctx *tmapi.Context, tx *transaction.Tr
state := registryState.NewMutableState(ctx.State())
switch tx.Method {
case MethodRegisterEntity:
var sigEnt entity.SignedEntity
if err := cbor.Unmarshal(tx.Body, &sigEnt); err != nil {
return err
}
return app.registerEntity(ctx, state, &sigEnt)
case MethodRegisterNode:
var sigNode node.MultiSignedNode
if err := cbor.Unmarshal(tx.Body, &sigNode); err != nil {
return err
}
return app.registerNode(ctx, state, &sigNode)
case MethodRegisterRuntime:
var rt Runtime
if err := cbor.Unmarshal(tx.Body, &rt); err != nil {
return err
}
return app.registerRuntime(ctx, state, &rt)
```

```go
default:
return fmt.Errorf("지원되지 않는 메서드: %s", tx.Method)
}
}
```

사용 예제

1. 엔티티 등록 예제

```go
package main

import (
"context"
"crypto/rand"
"github.com/oasisprotocol/oasis-core/go/common/entity"
"github.com/oasisprotocol/oasis-core/go/common/crypto/signature"
registryAPI "github.com/oasisprotocol/oasis-core/go/registry/api"
)

func registerEntity() error {
// 1. 엔티티 키 생성
entitySigner, err := signature.NewSigner(rand.Reader)
if err != nil {
```

```
return err
}
// 2. 엔티티 디스크립터 생성
ent := &entity.Entity{
Versioned: cbor.NewVersioned(entity.LatestDescriptorVersion),
ID: entitySigner.Public(),
Nodes: []signature.PublicKey{}, // 나중에 노드 추가
}
// 3. 엔티티 서명
sigEnt, err := entity.SignEntity(entitySigner, registryAPI.RegisterEntitySignatu
if err != nil {
return err
}
// 4. 등록 트랜잭션 생성
tx := registryAPI.NewRegisterEntityTx(0, nil, sigEnt)
// 5. 트랜잭션 제출 (실제 구현에서는 합의 서비스 사용)
return submitTransaction(tx)
}
```

2. 노드 등록 예제

```
func registerNode(entitySigner signature.Signer) error {
// 1. 노드 키들 생성
```

```
nodeSigner, _ := signature.NewSigner(rand.Reader)

p2pSigner, _ := signature.NewSigner(rand.Reader)

consensusSigner, _ := signature.NewSigner(rand.Reader)

tlsSigner, _ := signature.NewSigner(rand.Reader)

// 2. 노드 디스크립터 생성

n := &node.Node{

Versioned: cbor.NewVersioned(node.LatestNodeDescriptorVersion),

ID: nodeSigner.Public(),

EntityID: entitySigner.Public(),

Expiration: getCurrentEpoch() + 100, // 100 에포크 후 만료

TLS: node.TLSInfo{

PubKey: tlsSigner.Public(),

},

P2P: node.P2PInfo{

ID: p2pSigner.Public(),

Addresses: []node.Address{

{

TCPAddr: node.TCPAddr{

IP: net.ParseIP("127.0.0.1"),

Port: 9200,

},

},

},
```

```
},
Consensus: node.ConsensusInfo{
ID: consensusSigner.Public(),
Addresses: []node.ConsensusAddress{
{
ID: consensusSigner.Public(),
Address: node.Address{
TCPAddr: node.TCPAddr{
IP: net.ParseIP("127.0.0.1"),
Port: 9201,
},
},
},
},
},
Roles: node.RoleValidator | node.RoleComputeWorker,
}
// 3. 노드 서명 (엔티티와 노드 키로 다중 서명)
sigNode, err := node.MultiSignNode(
[]signature.Signer{entitySigner, nodeSigner},
registryAPI.RegisterNodeSignatureContext,
n,
)
```

```go
if err != nil {
return err
}
// 4. 등록 트랜잭션 생성 및 제출
tx := registryAPI.NewRegisterNodeTx(0, nil, sigNode)
return submitTransaction(tx)
}
```

3. 런타임 등록 예제

```go
func registerRuntime(entitySigner signature.Signer) error {
// 1. 런타임 디스크립터 생성
rt := &registryAPI.Runtime{
Versioned: cbor.NewVersioned(registryAPI.LatestRuntimeDescriptorVersion),
ID: common.NewTestNamespaceFromSeed([]byte("test runtime"), 0),
EntityID: entitySigner.Public(),
Kind: registryAPI.KindCompute,
TEEHardware: node.TEEHardwareIntelSGX,
Deployments: []*registryAPI.VersionInfo{
{
Version: version.Version{Major: 1, Minor: 0, Patch: 0},
TEE: []byte("SGX enclave hash"),
},
```

```
},
Executor: registryAPI.ExecutorParameters{
GroupSize: 3,
GroupBackupSize: 1,
AllowedStragglers: 1,
RoundTimeout: 30 * time.Second,
MaxMessages: 128,
},
TxnScheduler: registryAPI.TxnSchedulerParameters{
Algorithm: registryAPI.TxnSchedulerSimple,
BatchFlushTimeout: 1 * time.Second,
MaxBatchSize: 1000,
MaxBatchSizeBytes: 16 * 1024 * 1024, // 16MB
},
Storage: registryAPI.StorageParameters{
GroupSize: 3,
MinWriteReplication: 2,
MaxApplyWriteLogEntries: 100000,
MaxApplyOps: 2,
MaxMergeRoots: 8,
MaxMergeOps: 2,
},
AdmissionPolicy: registryAPI.RuntimeAdmissionPolicy{
```

```
AnyNode: &registryAPI.AnyNodeRuntimeAdmissionPolicy{},
},
GovernanceModel: registryAPI.GovernanceEntity,
}
// 2. 등록 트랜잭션 생성 및 제출
tx := registryAPI.NewRegisterRuntimeTx(0, nil, rt)
return submitTransaction(tx)
}
```

모니터링과 관리

1. 상태 조회

```
// 등록된 모든 엔티티 조회
func listEntities(registry registryAPI.Backend) error {
ctx := context.Background()
entities, err := registry.GetEntities(ctx, consensusAPI.HeightLatest)
if err != nil {
return err
}
for _, ent := range entities {
fmt.Printf("엔티티 ID: %s, 노드 수: %d\n",
ent.ID, len(ent.Nodes))
```

```
}
return nil
}

// 특정 런타임의 노드 조회
func getNodesForRuntime(registry registryAPI.Backend, runtimeID common.Namespac
ctx := context.Background()
nodes, err := registry.GetNodes(ctx, consensusAPI.HeightLatest)
if err != nil {
return err
}
for _, n := range nodes {
for _, rt := range n.Runtimes {
if rt.ID.Equal(&runtimeID) {
fmt.Printf("노드 %s가 런타임 %s를 지원함\n",
n.ID, runtimeID)
break
}
}
}
return nil
}
```

2. 이벤트 모니터링

```
func watchRegistryEvents(registry registryAPI.Backend) error {
ctx := context.Background()
// 엔티티 이벤트 모니터링
entityCh, entitySub, err := registry.WatchEntities(ctx)
if err != nil {
return err
}
defer entitySub.Close()
// 노드 이벤트 모니터링
nodeCh, nodeSub, err := registry.WatchNodes(ctx)
if err != nil {
return err
}
defer nodeSub.Close()
for {
select {
case ev := <-entityCh:
if ev.IsRegistration {
fmt.Printf("새 엔티티 등록: %s\n", ev.Entity.ID)
} else {
fmt.Printf("엔티티 업데이트: %s\n", ev.Entity.ID)
```

```
}
case  ev := <-nodeCh:
if  ev.IsRegistration {
fmt.Printf("새 노드 등록: %s (역할: %v)\n",
ev.Node.ID, ev.Node.Roles)
} else {
fmt.Printf("노드 업데이트: %s\n", ev.Node.ID)
}
case <-ctx.Done():
return  ctx.Err()
}
}
}
```

보안 고려사항

1. 서명 검증

- 모든 등록은 적절한 키로 서명되어야 함
- 엔티티 키와 노드 키의 다중 서명 검증
- 서명 컨텍스트를 통한 도메인 분리

2. 권한 관리

- 엔티티만 자신의 노드를 등록할 수 있음

- 런타임 등록은 해당 엔티티만 가능
- 스테이킹 요구사항을 통한 스팸 방지

3. 상태 일관성

- 트랜잭션 원자성 보장
- 상태 변경 시 적절한 검증
- 이벤트를 통한 상태 변경 알림

이러한 레지스트리 서비스를 통해 Oasis Core는 네트워크 참여자들의

신원과 메타데이터를 안전하고 효율적으로 관리할 수 있습니다.